

```

# Guardrails Governance Recommendation

## Current State: What Exists

### Layer 1: Pre-commit Hook (`.husky/pre-commit`)
``,`
AI Code Review (DeepSeek) → TypeScript Check → Unit Tests
``,`
- **Code Review**: Hardcoded to `deepseek-chat` model via
  `api.deepseek.com`
- **Reviewer prompt**: Generic "senior code reviewer" – no role context
- **Tests**: `run-smart-tests.cjs` maps changed files → relevant test
  files; system/config changes trigger full `npm test`
- **Blocking**: `CHANGES_REQUIRED` from AI review → commit blocked; test
  failures → commit blocked

### Layer 2: Pre-push Hook (`.husky/pre-push`)
- Blocks direct pushes to `develop` and `main`

### Layer 3: GitHub Actions CI (on PR to develop/qa/staging/main)
8 required status checks:
1. Secrets & Credentials Scan
2. Build, Lint, TypeCheck & Test (Vitest unit + API contract +
  integration)
3. Supabase Migration Dry-Run
4. CI Summary
5. SonarQube Cloud Analysis
6. Reviewdog Lint Check
7. AI-Powered PR Review (PR-Agent with DeepSeek)
8. CodeQL

### Layer 4: Branch Protection on `develop`
- 8 status checks required
- 1 approving review required (cannot self-approve)
- `enforce_admins: true` – even admins subject to rules
- `dismiss_stale_reviews: true`
- `require_code_owner_reviews: true`

### Layer 5: AGENTS.md Mandate
- Feature branch → PR → CI → code review → merge (mandatory for all
  agents)
- Pre-commit gate: AI review → TypeScript check → unit tests
- Compliance checklists in STATE_MATRIX.json

---

## Problems Identified

### P1: One-Size-Fits-All Validation
**Same rules apply to:**
- Agent 07a (UI Agent) building product features
- Agent 06 (DevOps) writing Docker/K8s configs
- Agent 10 (Tech Pub) writing documentation
- Agent 12 (Marketing) creating campaign assets

```

- AI Assistant (opencode) doing session work / internal tooling
- Agent 13 (Maintenance) applying hotfixes

****Reality:**** Product tests (Vitest unit, API contract, integration, E2E) are relevant to Agents 07a-07d and 08, but irrelevant to Agents 06, 10, 12, and internal tooling work.

P2: Hardcoded Code Reviewer

- ``code-review.cjs`` always uses ``deepseek-chat`` with a generic "senior code reviewer" prompt
- No role-specific context (e.g., UI agent should be reviewed by "Senior Frontend Engineer", not generic reviewer)
- When ``DEEPSEEK_API_KEY`` is not set, review is skipped (but this shouldn't block anything)
- The PR-Agent in GitHub Actions also uses DeepSeek with no role context

P3: No Test Tiering

- ``run-smart-tests.cjs`` maps files → tests, but doesn't know which agent is committing
- System/config file changes (``00_state_ledger/``, ``.github/``, ``AGENTS.md``) trigger FULL test suite – this blocks non-product work
- No mechanism for agents to declare "I don't need tests for this commit"

P4: CI Pipeline Blocks Non-Product Work

- ``ci-feature-push.yml`` runs ``npm test`` on every feature branch push
- If any test fails (even unrelated), the feature branch CI is red
- This means a DevOps PR modifying Docker configs can't merge because an unrelated API test fails

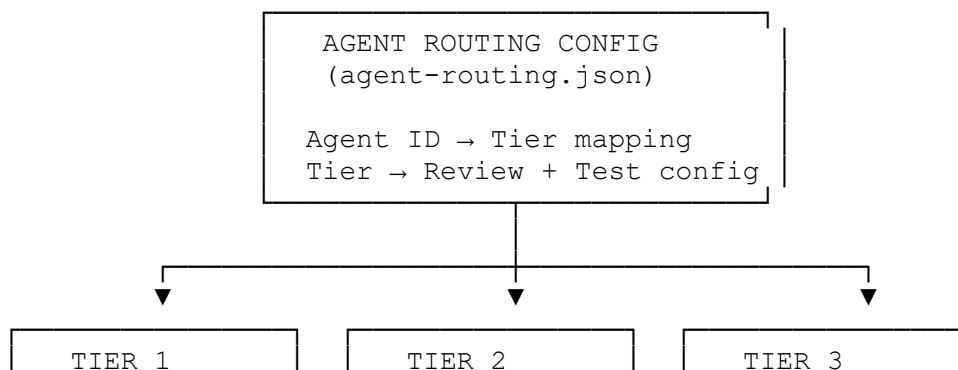
P5: Session Work Gets Same Treatment

- When the AI assistant works in a session (coding, refactoring, tooling), it faces the same gates as product agents
- This is correct for the feature-branch workflow, but tests should be optional for non-product changes

Recommendation: Tiered Governance System

Architecture

...



PRODUCT	INFRA/TOOLING	DOCUMENTATION
Agents: 07a,07b,07c, 07d, 08, 09	Agents: 06, 14, 15, AI Assistant	Agents: 10, 12, 11
Review: Role-specific AI review + PR review	Review: Role-specific AI review	Review: Skip or minimal review
Tests: Full product suite	Tests: TypeScript only	Tests: Skip
CI: All 8 checks	CI: Build+TypeCheck No test gate	CI: Build only (no tests)

...

Change 1: Agent Routing Configuration

New file: `.husky/agent-routing.json`

```
```json
{
 "tiers": {
 "product": {
 "description": "Full product validation - code review + TypeScript
+ tests",
 "agents": ["07a", "07b", "07c", "07d", "08", "09"],
 "reviewer_roles": {
 "07a": "Senior Frontend Engineer / UI Architect",
 "07b": "Senior Backend Engineer / API Architect",
 "07c": "Senior Database Engineer / DBA",
 "07d": "Senior Integration Engineer / Systems Architect",
 "08": "Senior QA Architect / Test Manager",
 "09": "Senior Release Manager / DevOps Lead"
 },
 "pre_commit": {
 "ai_review": true,
 "typescript_check": true,
 "tests": true,
 "test_command": "npm test"
 },
 "ci_checks": ["all"]
 },
 "infra": {
 "description": "Infrastructure and tooling - code review +
TypeScript, tests optional",
 "agents": ["06", "14", "15"],
 "session_work": true,
 "reviewer_roles": {
 "06": "Senior DevOps Architect / Platform Engineer",
```

```

 "14": "Senior FinOps Analyst / Cloud Cost Lead",
 "15": "Senior Security Engineer / Compliance Officer",
 "session": "Senior Software Engineer"
 },
 "pre_commit": {
 "ai_review": true,
 "typescript_check": true,
 "tests": false
 },
 "ci_checks": ["secrets-scan", "build", "typescript", "ci-summary"]
},
"docs": {
 "description": "Documentation and marketing – minimal validation",
 "agents": ["10", "11", "12"],
 "reviewer_roles": {
 "10": "Senior Technical Writer / Documentation Lead",
 "11": "Senior Customer Success Manager / Operations Lead",
 "12": "Senior Marketing Manager / Growth Lead"
 },
 "pre_commit": {
 "ai_review": false,
 "typescript_check": false,
 "tests": false
 },
 "ci_checks": ["secrets-scan", "ci-summary"]
}
},
"file_overrides": {
 "patterns": [
 {
 "match": "app/agent-ic-console/**",
 "tier_override": "infra",
 "reason": "Internal tooling – not product code"
 },
 {
 "match": "00_state_ledger/**",
 "tier_override": "infra",
 "reason": "State management – no product tests needed"
 },
 {
 "match": "docs/**",
 "tier_override": "docs",
 "reason": "Documentation only"
 },
 {
 "match": "agents/**",
 "tier_override": "docs",
 "reason": "Agent deliverables – not product code"
 },
 {
 "match": "context_lake/**",
 "tier_override": "docs",
 "reason": "Context lake – not product code"
 }
],

```

```

 {
 "match": "scripts/**",
 "tier_override": "infra",
 "reason": "Scripts and tooling"
 },
 {
 "match": "CHANGELOG.md",
 "tier_override": "docs",
 "reason": "Documentation"
 }
]
},
"role_based_review": {
 "enabled": true,
 "provider": "deepseek",
 "endpoint": "https://api.deepseek.com/chat/completions",
 "model": "deepseek-chat",
 "prompt_template": "You are a {role}. Review this git diff for: logic errors, security issues, bugs, code quality, and adherence to project requirements for your domain expertise. Start with APPROVED or CHANGES_REQUIRED on the first line, then provide specific feedback with file paths and line numbers if issues found."
}
}
...

```

### ### Change 2: Role-Based Code Review

```

Current (`code-review.cjs` line 38):
```javascript
{ role: 'system', content: 'You are a senior code reviewer...' }
```

Proposed:
```javascript
// Read agent-routing.json
const routing = require('./agent-routing.json');
const agentTier = detectAgentTier(changedFiles, routing);
const reviewerRole = getReviewerRole(agentTier, routing);

const prompt = routing.role_based_review.prompt_template
  .replace('{role}', reviewerRole);

// Use role-specific prompt
{ role: 'system', content: prompt }
```

How agent detection works:
1. Check `git diff --cached --name-only` for file paths
2. Match against `file_overrides.patterns` first (e.g., `app/agentic-console/**` → infra tier)
3. If no file override matches, check the commit message for agent ID pattern (e.g., `[Agent 07a]` or `07a_ui_agent`)

```

4. If neither matches, default to `infra` tier (safe default – review + TypeScript, no tests)

### ### Change 3: Smart Test Selection by Tier

```
Current (`run-smart-tests.cjs`):
```javascript  
// System/config files → full test suite (too aggressive)  
if (systemFiles.length > 0) {  
  execSync('npm test 2>&1', ...); // BLOCKS non-product work  
}  
```.  

Proposed:
```javascript  
const routing = require('./agent-routing.json');  
const agentTier = detectAgentTier(changedFiles, routing);  
const tierConfig = routing.tiers[agentTier];  
  
if (!tierConfig.pre_commit.tests) {  
  console.log(` ✓ Tests skipped: ${agentTier} tier  
(${tierConfig.description})`);  
  process.exit(0);  
}  
  
// Run tier-specific test command  
execSync(tierConfig.pre_commit.test_command + ' 2>&1', ...);  
```
```

### ### Change 4: CI Workflow Tiering

**\*\*Current:\*\*** `ci-feature-push.yml` runs `npm test` for ALL feature branches.

**\*\*Proposed:\*\*** Add a conditional job matrix:

```
```yaml  
jobs:  
  detect-tier:  
    runs-on: ubuntu-latest  
    outputs:  
      tier: ${ steps.detect.outputs.tier }  
    steps:  
      - uses: actions/checkout@v4  
      - id: detect  
        run: |  
          # Detect tier from changed files or commit message  
          TIER=$(node .husky/detect-tier.js)  
          echo "tier=$TIER" >> $GITHUB_OUTPUT  
  
  product-tests:  
    needs: detect-tier  
    if: needs.detect-tier.outputs.tier == 'product'  
    runs-on: ubuntu-latest
```

```

    steps:
      - run: npm test # Full product test suite

infra-checks:
  needs: detect-tier
  if: needs.detect-tier.outputs.tier == 'infra'
  runs-on: ubuntu-latest
  steps:
    - run: npx tsc --noEmit # TypeScript only

docs-check:
  needs: detect-tier
  if: needs.detect-tier.outputs.tier == 'docs'
  runs-on: ubuntu-latest
  steps:
    - run: echo "Documentation-only PR – no code checks needed"
...

```

Change 5: Branch Protection Relaxation

****Current:**** 8 required status checks on `develop` – ALL must pass.

****Proposed:**** Reduce to 3 universal checks + conditional tier checks:

****Universal (always required):****

1. `Secrets & Credentials Scan` – security baseline
2. `CI Summary` – audit trail
3. `CodeQL` – security scanning

****Conditional (required only when tier matches):****

4. `Build, Lint, TypeCheck & Test` – required for `product` tier only
5. `AI-Powered PR Review` – required for `product` and `infra` tiers
6. `Supabase Migration Dry-Run` – required only when
`supabase/migrations/` changes
7. `SonarQube Cloud Analysis` – required for `product` tier only
8. `Reviewdog Lint Check` – required for `product` tier only

****Implementation:**** GitHub branch protection rules don't support conditional checks natively. Two options:

****Option A (Recommended):** Separate branch protection rules per tier**

- Keep `develop` protection with minimal checks (secrets, CodeQL, CI summary)
- Add `required\_status\_checks.contexts` dynamically via API when PR is opened
- Use GitHub Rulesets (newer API) for conditional enforcement

****Option B: Use `if:` conditions in workflow jobs****

- Keep all 8 status checks in branch protection
- Make each job `if: needs.detect-tier.outputs.tier == 'product'` (or skip)
- When skipped, the check shows as "neutral" (not required) – but branch protection still requires it

- ****Problem:**** GitHub treats skipped checks as "pending" not "passed", so this doesn't work with required status checks

****Option A is the only viable path.**** Here's how:

1. Remove branch protection from `develop` entirely
2. Create a GitHub App (or use PAT) that:
 - Listens for PR opened/synchronize events
 - Detects tier from changed files
 - Uses GitHub API to dynamically set required status checks on the PR's head branch
 - Or: uses GitHub Rulesets API to conditionally enforce

****Simpler alternative:**** Use a bot/automation that:

- On PR open: adds required checks as commit statuses (pass/fake)
- On PR close: removes them
- This is hacky but works with branch protection

Change 6: Agent Compliance Checklist Update

****Current:**** Every agent has identical compliance checks (feature_branch_used, pr_created, ci_pipeline_passed).

****Proposed:**** Add tier-aware compliance checks:

```
```json
{
 "id": "UI-013",
 "check": "ci_checks_appropriate_for_tier() - CI ran the correct subset
of checks based on agent tier",
 "type": "workflow",
 "required": true,
 "passed": true,
 "tier": "product"
}
```
```

For infra agents:

```
```json
{
 "id": "DEVOPS-007",
 "check": "ci_checks_appropriate_for_tier() - CI ran infra-appropriate
checks (TypeScript, no product tests)",
 "type": "workflow",
 "required": true,
 "passed": true,
 "tier": "infra"
}
```
```

Implementation Priority


```
### Phase 1: Quick Wins (Immediate – No infrastructure changes)
1. **Update `code-review.cjs`** to read reviewer role from agent-
routing.json
2. **Update `run-smart-tests.cjs`** to skip tests for infra/docs tiers
3. **Create `.husky/agent-routing.json`** with tier definitions and file
overrides
4. **Update `ci-feature-push.yml`** to detect tier and conditionally run
tests
5. **Remove `--no-verify` workaround** from AGENTS.md since tests are now
tier-aware
```

```
### Phase 2: Branch Protection (Requires GitHub API automation)
1. **Relax `develop` branch protection** to minimal checks
2. **Create tier-aware PR check automation** (GitHub Actions workflow or
bot)
3. **Update AGENTS.md** governance rules to reference tiers
```

```
### Phase 3: Full Integration (Agent 00 Supervisor integration)
1. **Agent 0 reads agent-routing.json** when auditing compliance
2. **STATE_MATRIX.json** includes tier per agent
3. **Supervisor dashboard** shows tier-aware compliance status
```

GitHub-Native Test Selection Options

```
### Option 1: `paths` / `paths-ignore` in Workflow Triggers (RECOMMENDED
– Simplest)
```

GitHub Actions workflows support `paths` filters that **automatically skip** the entire workflow (or specific jobs) when only certain files change.

Example: Skip product tests when only docs/infra files change:

```
` `yaml
# .github/workflows/ci-feature-push.yml
name: CI – Feature Branch Gate

on:
  push:
    branches-ignore: [develop, qa, staging, main]
    paths-ignore:
      - 'docs/**'
      - 'agents/**'
      - 'context_lake/**'
      - '00_state_ledger/**'
      - 'app/agent-ic-console/**'
      - 'scripts/**'
      - '*.md'
      - 'CHANGELOG.md'
```

****Result:**** If you only change ``docs/`` or ``app/agent-ic-console/``, the entire CI workflow is ****skipped**** – no tests, no build, nothing. The PR shows "No jobs ran" instead of failing.

****For jobs WITHIN a workflow:****

``yaml`

```
jobs:
  detect-changes:
    runs-on: ubuntu-latest
    outputs:
      has-product-code: ${ steps.filter.outputs.product }
      has-infra-code: ${ steps.filter.outputs.infra }
    steps:
      - uses: actions/checkout@v4
      - uses: dorny/paths-filter@v3
        id: filter
        with:
          filters: |
            product:
              - 'app/**'
              - 'lib/**'
              - 'components/**'
              - '__tests__/**'
              - 'e2e/**'
              - 'supabase/migrations/**'
            infra:
              - '.husky/**'
              - '.github/**'
              - 'scripts/**'
              - '00_state_ledger/**'

  product-tests:
    needs: detect-changes
    if: needs.detect-changes.outputs.has-product-code == 'true'
    runs-on: ubuntu-latest
    steps:
      - run: npm test

  typescript-only:
    needs: detect-changes
    if: needs.detect-changes.outputs.has-product-code != 'true' &&
needs.detect-changes.outputs.has-infra-code == 'true'
    runs-on: ubuntu-latest
    steps:
      - run: npx tsc --noEmit

  docs-check:
    needs: detect-changes
    if: needs.detect-changes.outputs.has-product-code != 'true' &&
needs.detect-changes.outputs.has-infra-code != 'true'
    runs-on: ubuntu-latest
    steps:
      - run: echo "Docs-only changes – no code checks needed"
```

...

Option 2: PR Labels for Manual Test Control

Use GitHub PR labels to explicitly control which tests run. Maintainers or AI assistant can add labels when creating the PR.

****Labels to create:****

- `tests:full` - Run full product test suite
- `tests:typescript-only` - Run TypeScript check only
- `tests:skip` - Skip all tests
- `review:required` - Code review required
- `review:skip` - Skip code review

****Workflow implementation:****

```yaml

```
jobs:
 test-selection:
 runs-on: ubuntu-latest
 outputs:
 run-tests: ${ steps.check.outputs.run-tests }
 test-command: ${ steps.check.outputs.test-command }
 steps:
 - id: check
 run: |
 LABELS="${ github.event.pull_request.labels.*.name }"
 if echo "$LABELS" | grep -q "tests:skip"; then
 echo "run-tests=false" >> $GITHUB_OUTPUT
 echo "Tests skipped via label"
 elif echo "$LABELS" | grep -q "tests:typescript-only"; then
 echo "run-tests=true" >> $GITHUB_OUTPUT
 echo "test-command=npx tsc --noEmit" >> $GITHUB_OUTPUT
 else
 echo "run-tests=true" >> $GITHUB_OUTPUT
 echo "test-command=npm test" >> $GITHUB_OUTPUT
 fi

 run-tests:
 needs: test-selection
 if: needs.test-selection.outputs.run-tests == 'true'
 runs-on: ubuntu-latest
 steps:
 - run: ${ needs.test-selection.outputs.test-command }
```

...

**\*\*How it works in practice:\*\***

1. AI assistant creates PR with label `tests:skip` for agentic console changes
2. GitHub Actions reads the label and skips the test job
3. Branch protection sees the test job as "skipped" (neutral) - but with `required\_status\_checks`, this is tricky (see below)

### ### Option 3: GitHub Rulesets (Newer API – More Flexible Than Branch Protection)

GitHub Rulesets (available since late 2023) are more powerful than traditional branch protection:

```
```json
{
  "name": "Product Branch Rules",
  "target": "branch",
  "enforcement": "active",
  "conditions": {
    "ref_name": {
      "include": ["refs/heads/develop"]
    }
  },
  "bypass_actors": [],
  "rules": [
    {
      "type": "required_status_checks",
      "parameters": {
        "required_status_checks": [
          {"context": "Secrets & Credentials Scan"},
          {"context": "CodeQL"},
          {"context": "CI Summary"}
        ],
        "strict_required_status_checks_policy": false
      }
    }
  ]
}
```
```

**\*\*Limitation:\*\*** Rulesets still don't support conditional checks based on PR labels or file paths natively. But they do support:

- `bypass\_actors` – Allow specific teams/users to bypass rules
- `additional\_required\_checks` – Add checks dynamically

### ### Option 4: Custom GitHub App / Bot (Most Powerful)

For full per-PR test control, create a lightweight GitHub App that:

1. Listens for `pull\_request.opened` and `pull\_request.synchronize` events
2. Reads a `.github/test-config.json` file from the PR's head branch:

```
```json
{
  "tests": "skip",
  "review": "required",
  "reason": "Internal tooling – no product tests needed"
}
```
```

3. Uses GitHub API to:
  - Add/remove labels

- Set commit statuses
- Update required checks on the branch

**\*\*This is the most flexible but requires hosting a bot.\*\***

---

**## Recommended Approach: Combine Options 1 + 2**

**### For CI Workflows (Automatic): Use `paths` filters**

- `ci-feature-push.yml` uses `paths-ignore` to skip entirely for docs/infra changes
- `ci.yml` (PR to develop) uses `dorny/paths-filter` to conditionally run tests

**### For Branch Protection (Manual Override): Use PR Labels**

- Add `tests:skip` or `tests:typescript-only` labels
- Modify branch protection to check a "tier-gate" status check that passes based on labels
- The tier-gate workflow runs first, then conditionally triggers test jobs

**### Implementation in Branch Protection**

Instead of requiring `Build, Lint, TypeCheck & Test` directly, require a **\*\*wrapper check\*\*** called `tier-gate`:

```

` ``yaml
.github/workflows/tier-gate.yml
name: Tier Gate

on:
 pull_request:
 branches: [develop]

jobs:
 gate:
 name: Tier Gate
 runs-on: ubuntu-latest
 steps:
 - uses: actions/checkout@v4
 - uses: dorny/paths-filter@v3
 id: filter
 with:
 filters: |
 product:
 - 'app/(customer|admin|merchant)/**'
 - 'lib/**'
 - 'components/**'
 - '___tests___/**'
 - 'e2e/**'

 - name: Set gate status
 run: |

```

```

 if ["${{ steps.filter.outputs.product }}" == "true"]; then
 echo "## Tier: PRODUCT - Full test suite required" >>
$GITHUB_STEP_SUMMARY
 echo "status=pending" >> $GITHUB_OUTPUT
 elif [-n "${{ github.event.pull_request.labels }}"] && echo
"${{ github.event.pull_request.labels.*.name }}" | grep -q "tests:skip";
then
 echo "## Tier: SKIPPED - Tests skipped via label" >>
$GITHUB_STEP_SUMMARY
 echo "status=success" >> $GITHUB_OUTPUT
 else
 echo "## Tier: INFRA - TypeScript check only" >>
$GITHUB_STEP_SUMMARY
 echo "status=success" >> $GITHUB_OUTPUT
 fi

- name: Check gate
 run: |
 if ["${{ steps.check.outputs.status }}" == "pending"]; then
 echo "Full tests required for product changes"
 exit 1
 fi
 echo "Gate passed"
...

```

**\*\*Branch protection then requires only:\*\***

1. `Secrets & Credentials Scan`
2. `Tier Gate` ← This is the smart check that routes to the right test suite
3. `CodeQL`
4. `CI Summary`

The `Tier Gate` check either passes immediately (infra/docs) or fails and triggers the full test suite (product). This way, branch protection has a single check that adapts to the change type.

---

#### ## Key Decisions Needed

1. **\*\*Should the tier be determined by file path or agent ID?\*\***  
 - Recommendation: File path first (matches what's actually changed), agent ID as fallback
2. **\*\*Should the AI code review be blocking for infra/docs tiers?\*\***  
 - Recommendation: Non-blocking (informational only) for infra; skip entirely for docs
3. **\*\*Should branch protection be reduced or made tier-aware?\*\***  
 - Recommendation: Reduce to 3-4 universal checks; use `Tier Gate` wrapper check
4. **\*\*Should session work (AI assistant) use the infra tier by default?\*\***

- Recommendation: Yes – AI assistant does a mix of product and infra work; file-path detection handles routing

5. **\*\*Who determines the tier – file paths, PR labels, or commit message?\*\***

- Recommendation: File paths (automatic via `paths-filter`) + PR labels (manual override for edge cases)

6. **\*\*Can we have per-PR test selection?\*\***

- Yes – via PR labels (`tests:skip`, `tests:typescript-only`, `tests:full`)
- Combined with `paths` filters for automatic detection
- Branch protection sees a single `Tier Gate` check that adapts